

上下文即操作系统

Agent Context Management

本文沿着四条线索展开上下文管理系统的设计：DeepSeek Harness 的产线级实现、MemGPT 把操作系统虚拟内存分层搬进 LLM 的思路、Context-Folding 让模型学会「折叠」上下文的训练范式、ACON 用失败案例反推最优压缩规则的工程方法——一直追到工业界的实战套路，核心要回答的问题只有一个：**如果你要构建一个领先的 Agent 上下文管理系统，应该怎么做。**

00 上下文工程：Agent 与 LLM 之间的桥梁

在进入 DeepSeek Harness、MemGPT、Context-Folding 等具体实现之前，先回答一个更根本的问题：上下文工程到底是什么，为什么它是 Agent 工程中最困难、也最决定成败的部分。本章为全篇奠定地基。

CORE THESIS

上下文是 LLM 与 Agent 之间的桥梁。它承载了记忆管理、工具加载、Skill 发现、打断重连等最核心的模块；桥梁的设计直接决定了一个系统能解决多复杂的问题，以及解决这些问题的开销。业界至今没有统一的最优解——因为「最优」取决于模型能力、任务形状、成本预算与延迟约束之间的取舍。

上下文工程 = Agent 与 LLM 之间的桥梁

上下文是直接 与 LLM 交互的部分，是 LLM 与 Agent 之间的桥梁；桥梁的设计决定了整个系统能够承载的负重



桥梁决定承载负重

上下文设计决定 Agent 能解决多复杂的问题，以及解决问题的开销：

KV cache 命中率—设计好的系统 > 90%，设计差的仅 ~30%，成本相差一倍不止

图 0-1 · 上下文工程 = Agent 与 LLM 之间的桥梁：承载的七大核心模块

源文件：context-diagrams/00-context-engineering-bridge.svg

0.1 什么是上下文工程

先区分两个容易被混为一谈的概念：

- **提示词工程 (Prompt Engineering)**：优化「单次怎么说」——写一条指令、一个 system prompt、几个 few-shot 示例。它聚焦在单次输入输出上。
- **上下文工程 (Context Engineering)**：管理「知道什么、何时知道、如何知道」——在 Agent 的每一步推理循环中，动态地为有限的上下文窗口装配最相关、最简洁、最高质量的信息子集。

先解释一个贯穿全文的术语：**上下文窗口 (context window)**——LLM 单次推理能「看到」的文本上限，本质是模型的工作记忆。它由 token（词元，模型处理文本的最小单位，中文一个汉字约占 1-2 个 token）计量。窗口有硬上限，且存在「中段注意力衰减」现象（详见 0.6 节）。

上下文工程的本质是**信息流的架构设计**，而不是「写提示词」。它回答的是一组系统性问题：这一步模型需要看到什么？哪些信息该进窗口、哪些该留在外部存储？信息的排列顺序如何（模型对开头、结尾、中段的注意力并不均匀）？当信息逼近上限时，先丢弃什么、保留什么？开发者在这里的角色，从「提示词雕琢者」转变为「信息流架构师」。

这个术语由 AI 研究者 Andrej Karpathy 推广，他把上下文工程描述为「在每一步 workflows 中，用恰好正确的信息填充上下文窗口的艺术与科学」。

0.2 为什么困难：业界没有统一的最优解

上下文工程的困难来自三点，它们叠加在一起，构成了它「没有银弹」的底色：

① 它是唯一的桥梁

上下文是模型与 Agent 交互的**唯一通道**。模型的每一步决策都依赖桥梁送过去的信息，Agent 的每一步动作结果又要通过桥梁送回模型。桥梁的任何设计缺陷，都会被「每一步」放大。

② 模块彼此耦合

记忆管理、工具加载、Skill 发现、打断重连、检索注入、压缩摘要、预算管理——这些模块不是独立的。压缩太激进会丢信息，太保守会超预算；记忆存太多稀释注意力，存太少则「失忆」。

③ 最优依赖四个变量

模型能力（强模型可自主管理记忆，弱模型需框架兜底）、任务形状（短任务 vs 长程）、成本预算、延迟约束。四个变量组合无穷，所以不存在放之四海皆准的方案。

正因为「最优」随问题形状而变，才会出现大量开源上下文管理插件——每一个都针对某一种问题形状做了特定取舍。这是末尾「开源上下文管理插件精读」一节要逐一精读它们源码的原因：不是比谁更好，而是看懂它们各自在哪个变量上做了让步。

0.3 上下文承载的核心模块

图 0-1 已经展示了桥梁上承载的七大模块，这里展开说明为什么每一个都「核心」到不可缺席：

模块	解决的问题	为什么核心
记忆管理	跨会话、跨任务累积知识	没有它，Agent 每次启动都是「失忆」状态，无法区分新老用户、无法复用上次结论
工具加载	让模型知道能调用什么	工具定义（schema）本身占用 token，加载哪些、不加载哪些是实打实的预算取舍
Skill 发现	检索并注入可复用能力	区别于原始工具，Skill 是「已封装的操作序列」，发现错了等于给模型递错工具
打断重连	中断后从检查点恢复	长任务被人工打断、崩溃、超时后，能否不丢状态地续跑，决定系统是否可信
检索注入	按需取回外部知识	RAG 的核心：把「全部知识」换成「相关知识」，避免窗口被无关内容稀释
压缩摘要	逼近上限时压缩历史	compaction 是长任务续命的唯一手段，压缩质量直接决定多轮后还剩多少真信息
预算管理	追踪 token 消耗	它是「仪表盘」，决定上述机制何时触发、以什么优先级触发

0.4 上下文决定能力边界与开销

上下文设计的好坏，直接决定两件事：**Agent 能解决多复杂的问题**，以及**解决这些问题的开销**。而衡量「设计好坏」最硬的一个指标，是 **KV cache 命中率**。

先解释术语：**KV cache（键值缓存）**是 Transformer 模型推理时的内部机制——对已经处理过的 token，模型会缓存其注意力计算产生的键（Key）与值（Value）向量。当后续请求的前缀与已缓存内容逐字一致时，模型可

以直接复用这些计算结果、跳过重复计算，这就是**前缀缓存（prefix caching）**。它直接决定「每一轮要重新算多少」。

- 设计好的系统：静态部分（system prompt、工具定义、历史前缀）在每一轮保持**字节级一致**，命中率可以做到 **90% 以上**。
- 设计差的系统：动态内容（时间戳、可变变量）被塞进前缀，导致缓存反复失效，命中率可能只有 **30% 左右**。

成本差异来自计价方式：缓存命中的 token 通常按原价的约 1/10 甚至更低计费。命中率 90% 与 30% 之间，输入侧的实际成本可相差**一倍以上**，极端情况下达数倍——这正是「上下文工程直接决定开销」的量化证据。

EVIDENCE

工业界已有实测数据佐证这一杠杆：LangChain 的 Deep Agents 跨三个厂商测试，prompt caching 把 token 成本降低了 **49%—80%**；一个真实案例仅通过「把动态变量移到提示词末尾、保持静态前缀稳定」，就把命中率从 **7% 提到 84%**、整体成本降 59%—70%。Manus 团队更是直言：「如果要我只选一个指标，KV cache 命中率是生产级 Agent 最重要的指标。」

LangChain Blog · Deep Agents Prompt Caching; DigitalOcean · Advanced Prompt Caching

这里隐含一个反直觉的事实：上下文设计得好，不仅更便宜，而且更「专注」——因为稳定的前缀意味着模型每轮看到的结构是稳定的，注意力不会被无谓地重新分配。KV cache 命中率因此既是成本指标，也是质量指标。

0.5 上下文压缩的方式：三类主流方案

承接 0.6：长程任务里上下文必然无限增长，压缩是唯一能「续命」的手段。这一节先把业界最流行的文本压缩方案按「作用层次」分成三类，讲清每类的原理、代表项目与量化结果。理解这三类，等于拿到一张「上下文压缩地图」——后面几章的精读（Context-Folding、ACON）都能在这张地图上定位。

上下文压缩的三类主流方案：作用在进模型之前，各有取舍

软 / 硬 / 摘要三类均作用在「送进模型之前」，按是否要微调目标模型、压缩比 / 损失程度取舍



图 0-2 · 上下文压缩的三类主流方案：软 / 硬 / 摘要均作用在「送进模型之前」

源文件：context-diagrams/00-context-compression-taxonomy.svg

① 软压缩（Soft Prompt） — 把提示压成可学习的向量

先解释术语：**软提示（soft prompt）** 指的**不是文字**，而是一组**可训练的向量**——让模型「学出」一段能替代原文的压缩表示，模型直接条件于这组向量生成。代表工作是斯坦福的 **Gist Tokens** (Mu et al., NeurIPS 2023)：训练时在提示末尾追加几个特殊 token (gist token)，并修改注意力机制，让这些 gist token 能「看到」原始提示、而后面的生成 token 只能「看到」gist token——逼模型把原始提示的信息浓缩进这几个 token 的激活里。推理时只喂这几个 gist token，丢掉原文，最高做到 **26x 压缩**而质量几乎无损。同一路线的还有 **ICAIE (In-context Autoencoder, 上下文自编码器)** 与 **AutoCompressor** (EMNLP 2023) ——后者递归地把文档分段压成「摘要向量」，能处理 30K token 的序列。

软压缩的优点是最省 token、几乎无损；但代价很硬：**必须微调目标模型本身**，换一个模型就要重训，对只能用 API 的黑盒模型（GPT、Claude 等）完全不可行。所以它目前停留在「研究要跟踪」的类别，尚未进入生产工具链。

② 硬压缩（Extractive） — 删冗余 token、保留原文

这是最通用、落地最多的一类。核心思想：**原文里的 token 信息量并不相等**，用一个小模型给每个 token 打分，删掉低分的、保留高分的。先解释打分依据——**困惑度（perplexity）**：表示小模型对一个 token 的「意外程

度」。模型早就料到（困惑度低）说明它冗余、可删；模型感到意外（困惑度高）说明它信息量大、要留。

这条线有四个里程碑。最早的 **Selective Context** (Li et al., 2023) 用「自信息」过滤词元，约 **50% 压缩**。微软的 **LLMLingua** (EMNLP 2023) 把它升级为「由粗到细 (coarse-to-fine)」三层：预算控制器先压冗余段 → 句子级过滤 → token 级迭代压缩，最高 **20x 压缩** 下 GSM8K 数学推理仅降 **1.5%**。**LLMLingua-2** (ACL 2024) 做了一次范式转变：不再算困惑度，而是把压缩重构为 **token 分类** 问题——用 GPT-4 批量压缩海量文本得到「保留 / 删除」标签（**数据蒸馏**，让大模型「手把手」教小模型怎么压），再训练一个 BERT 级编码器（340M，比 v1 的 7B 小 20 倍），速度提升 **3—6 倍**。**LongLLMLingua** 则针对长上下文做「问题感知压缩 + 文档重排序」，把重要信息挪到开头和结尾，4x 压缩下性能**不降反升 17.1%**——这是对 0.5 节「中段盲区 (lost in the middle)」的直接回应。

硬压缩的优点是通用、无需训练大模型、可接黑盒 API；缺点是**有损**——极端压缩比下会丢细节，不适合「必须逐字保留」的场景（法律条文、代码精确引用），且需要额外的小模型来打分。

③ 摘要式 (Generative) — 用 LLM 生成精炼摘要

不再删 token，而是让 LLM 「重写」出一段更短的新文本。最轻量的一类是 **观察遮蔽 (observation masking)**：Agent 每轮的「推理 / 行动 / 观察」三部分里，观察（工具输出、文件读取）往往占 80% 以上 token——把冗长的观察换成紧凑引用（保留推理与行动原文），即可减少 60—80% 且几乎不伤决策。这一招由 OpenHands 首先提出，现在 Cursor、Warp 的编程 Agent 都在用。

更进阶的是**学出来的摘要器**。微软的 **Acon** (Agent Context Optimization, 2025) 是代表：它用「全量上下文成功 / 压缩上下文失败」的配对轨迹，让一个强模型分析「为什么压缩后失败」，据此**迭代优化「压缩指南」**（一段告诉压缩器该怎么压的自然语言规则），再把优化后的 LLM 压缩器蒸馏进小模型。在 AppWorld、OfficeBench 等真实长程任务上，Acon 把峰值 token 降低 **26—54%**，性能几乎不损；蒸馏后仍保留 95% 精度。同类还有 **RECOMP**（在检索后、进模型前压缩文档）与 **Nano-Capsulator**（把提示压成自然语言「胶囊」）。

摘要式的优点是语义灵活、压缩比高；缺点是**语义可能漂移**——重写可能丢失关键数字或状态，且每次压缩都多一次 LLM 调用，本身有成本与延迟。

三类方案 · 一张地图

三类方案不互斥、通常组合使用——硬压缩负责日常去冗余、摘要式负责阶段性续命、软压缩作为模型原生的压缩能力嵌入推理。选型只看四个问题：**只能用黑盒 API 吗**（是→排除软压缩）、**能微调目标模型吗**（能→可上软压缩）、**任务是否长程**（是→摘要式是主力）、**是否需逐字保留**（是→有损的硬压缩要谨慎）。回到 0.4 的主线：压缩不仅省 token，更短的稳定前缀还能提升 KV cache 命中率——好的压缩是「省成本」与「提质量」的双赢。

0.6 长程任务：好的设计不偏移，差的设计答非所问

这是上下文工程质量最直观的分水岭，也直接呼应本章标题里的「桥梁」：

设计差：几轮后答非所问

三个典型失效模式：**中段盲区 (lost in the middle)** ——模型对长上下文的中段注意力显著衰减；**前提污染 (context poisoning)** ——早期的一个错误假设逐轮放大，第 6 步时系统已在错误前提上自信地执行；**噪声稀释 (context distraction)** ——无关历史淹没信号。

设计好：长程任务不偏移

把「当前该看的」与「可以暂存的」分开，把「该记住的」持久化到外部，把「会做的」沉淀为可复用技能。模型每轮看到的都是精炼、稳定、有序的信息，偏移自然被抑制。

EVIDENCE

研究侧的量化支撑：Databricks 研究发现模型准确率在约 **32K token** 处就开始下降，远早于百万级窗口上限；Chroma 2025 年对 18 个模型的测试显示，即使是最简单的任务，性能也随上下文增长而非均匀地衰减。结论一致：**更大的窗口不解决问题，更聪明的装配才解决问题。**

Databricks 长上下文研究 · Chroma Research 2025 (18 模型测试)

0.7 优秀的长程任务管理开源案例

承接 0.6，下面看两个在「长程任务不偏移」上做出示范性设计的开源项目，并给出流程架构图。

Voyager (MineDojo) — 技能库驱动的长程闭环

Voyager 是让 Agent 在 Minecraft 中自主长程探索的项目，其上下文管理的核心是**技能库 (Skill Library)**。核心逻辑（读自 `[[路径已隐藏]]`、`curriculum.py`）：

Voyager：技能库驱动(long-term)任务闭环

技能不是一次性产物，而是可检索、可复用的「程序性记忆」——随任务累积，Agent 越用越强

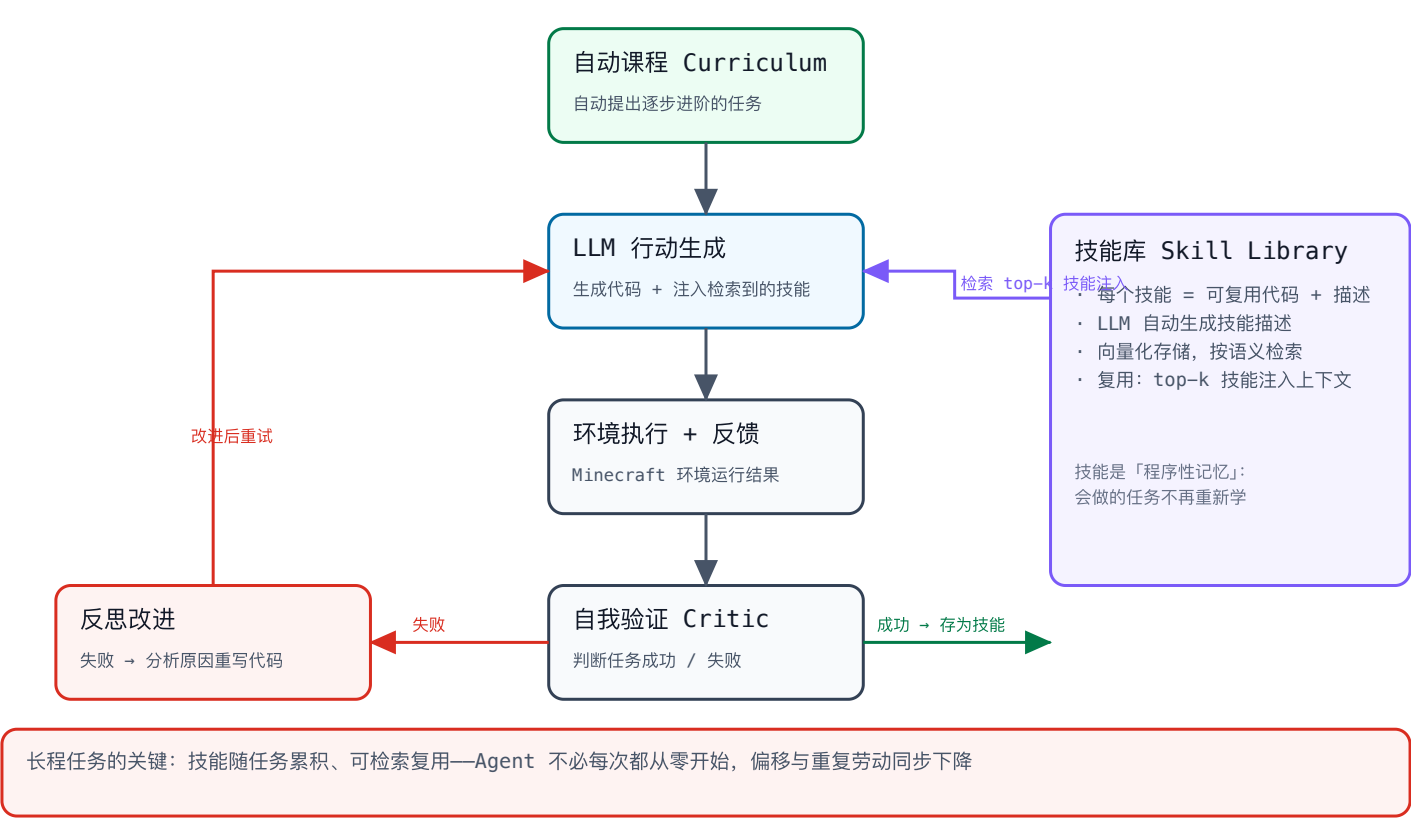


图 0-3 · Voyager：自动课程 → 行动生成 → 环境反馈 → 自我验证 → 成功沉淀为技能、失败反思重试的闭环
源文件：context-diagrams/00-voyager-skill-library.svg

关键在于「成功即沉淀」：SkillManager 在任务成功后，用 LLM 自动为代码生成一段描述，与代码一起向量化存入技能库；下一次遇到相似任务时，retrieve_skills() 按语义检索 top-k 技能并注入上下文。于是技能成为程序性记忆——会做的任务不再重新学。这让 Agent 在长程任务中技能不断累积、越用越强，同时避免了每次从零开始导致的偏移与重复劳动。其配套的 CurriculumAgent 维护已完成/失败任务清单，自动提出逐步进阶的新任务，构成外层的进度闭环。

A-MEM — 可演化的记忆网络

如果说 Voyager 解决的是「技能复用」，A-MEM 解决的则是「记忆结构自发演化」：

A-MEM: Zettelkasten 记忆演化三阶段

记忆不是只读的向量，而是可链接、可演化的知识网络——灵感来自卢曼的「卡片盒笔记法」

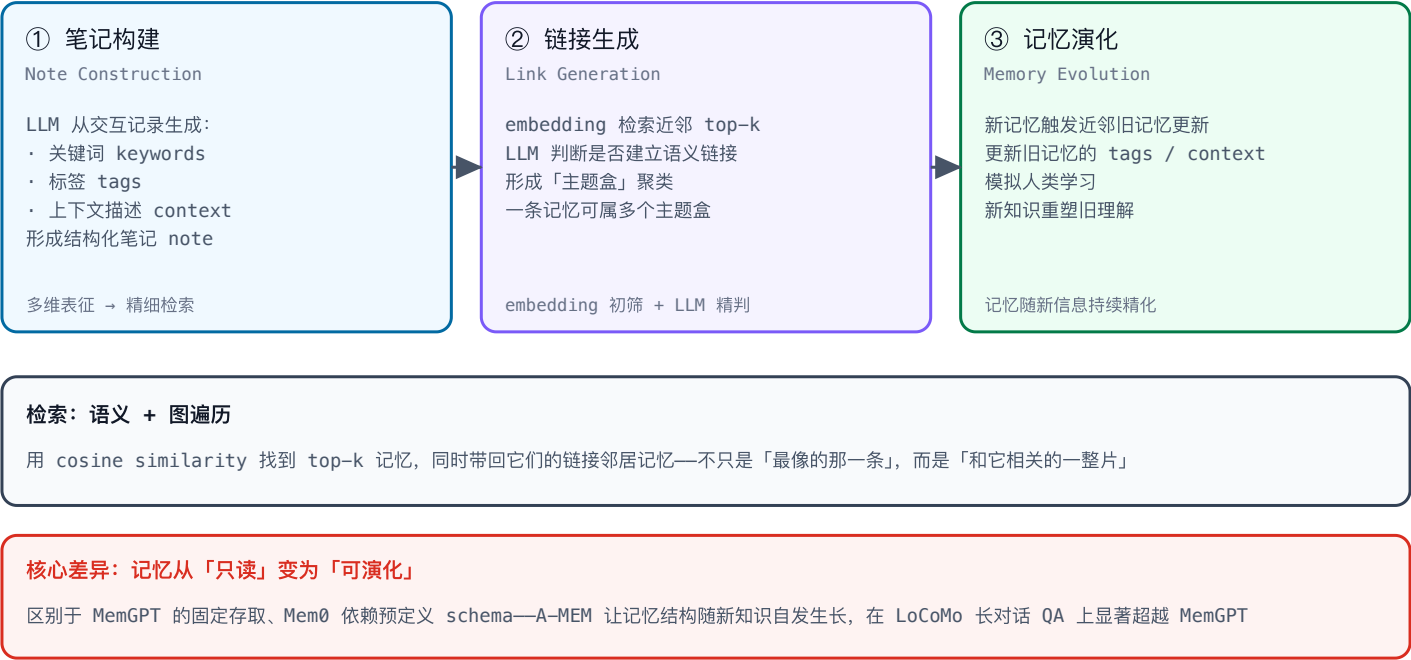


图 0-4 · A-MEM：笔记构建 → 链接生成 → 记忆演化三阶段，检索时带回链接邻居

源文件：context-diagrams/00-amem-zettelkasten.svg

它在 LoCoMo 长对话 QA 基准上显著超越 MemGPT。它的启示是：长程任务的记忆系统不只是「存得下」，更要「记得住彼此的关系、且能被新知识修正」——这也正是全篇后续要反复回到的主题：丢失的不是 token，而是结构。

本章小结 · 进入后续章节前的共识

上下文工程是 Agent 与 LLM 之间的桥梁，承载记忆、工具、Skill、打断重连、检索、压缩、预算七大模块；它决定能力边界与开销（KV cache 命中率是硬指标）；它没有统一最优解，只有针对特定问题形状的取舍。带着这四句话，再读后面的 dsh 源码精读、MemGPT/Letta、Context-Folding 与 ACON，你会看到它们各自是在哪个变量上做了让步、又补回了什么。

01 DeepSeek Harness 上下文管理全解

dsh 的上下文管理不是一个模块，而是一组分层的契约：日志层、投影层、事务层。精读自 `[[路径已隐藏]]`、`[[路径已隐藏]]`、`[[路径已隐藏]]` 源码。

CORE THESIS

上下文不是 `messages` 数组，而是「append-only 日志 + surface 投影 + 可回滚事务」的三层结构。所有压缩都是日志上的影蔽 (shadow)，不是删除。

1.1 三层架构：Log → Surface → Request

① Raw Session Event Log

append-only、lossless、永不删除。事件包括 `user/message`、`assistant/message+chunk`、`tool/call`、`tool/result`、`request/header`、`compaction/*` (仅日志)。崩溃后可完整重放。

② Surface (模型可见面)

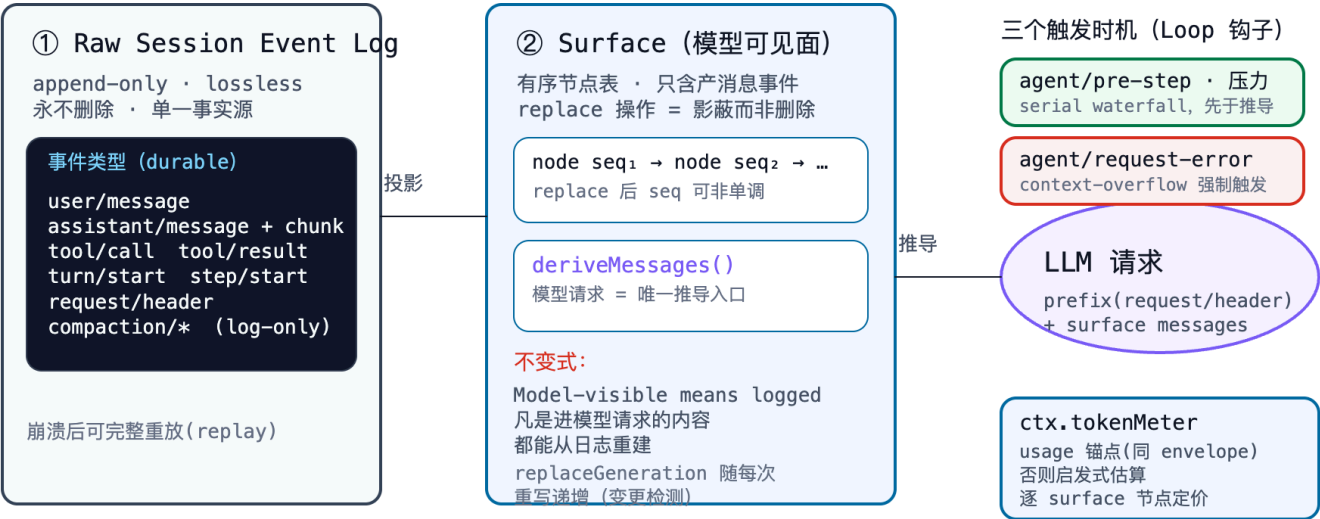
有序节点投影，只含产消息事件。`replace` 操作 = 影蔽而非删除；replace 后节点 seq 可非单调。`replaceGeneration` 随每次重写递增，用于变更检测。

③ deriveMessages()

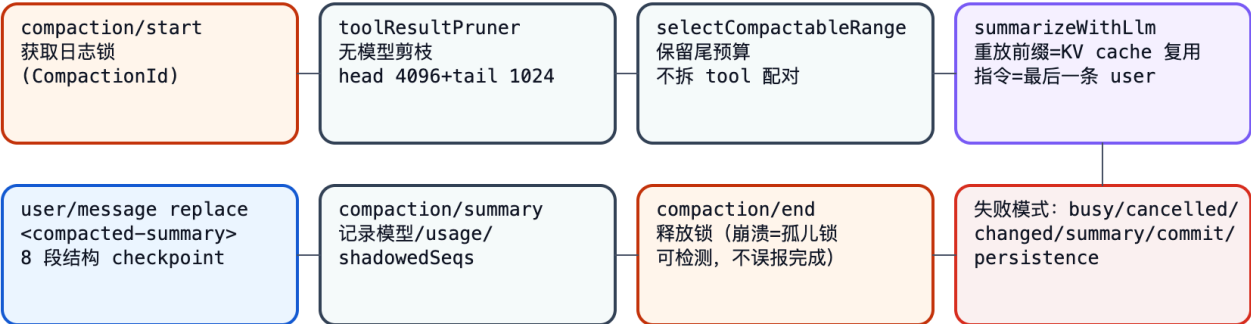
从 surface 增量投影出模型请求体的唯一入口。前端 `sessionMirror` 重放、回放测试都走它。没有「绕过日志直接拼 messages」的路径。

DeepSeek Harness 上下文管理 · 日志即真相，投影即上下文

论点：上下文不是 messages 数组，而是「append-only 日志 + surface 投影 + 可回滚事务」三层结构



③ Compaction 事务 (一次完整压缩的生命周期)



- 关键不变式:
- 1. 摘要必须严格小于被影蔽内容 (shrink 检查)
 - 2. 摘要调用复用会话自己的 system+tools+前缀 → 前缀缓存命中
 - 3. 边界必须 tool-call/result 配对平衡 (toolPairingBalanced)
 - 4. compaction/* 三事件只进日志不进 surface — 摘要搭乘 user/message

```
8 段 checkpoint 结构 (summarizer.ts)

## Primary Request and Intent
## Key Technical Concepts
## Files and Code
## Errors and Fixes
## Pending Jobs
## Current Work
## Next Step
## Critical Context
```

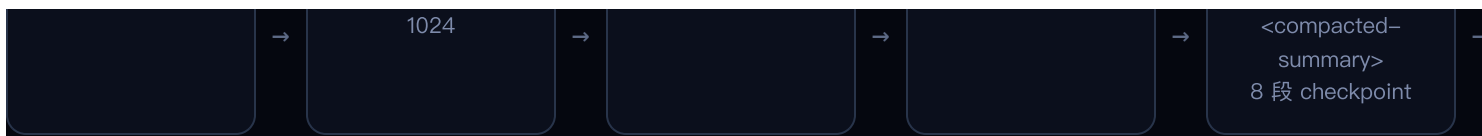
图 1 · DeepSeek Harness 上下文管理：三层结构 + Compaction 事务生命周期 (Excalidraw)

源文件：context-diagrams/01-dsh-context-management.excalidraw

1.2 Compaction 事务：一次压缩的完整生命周期

压缩在 dsh 中是一个受日志保护的事务，四阶段被 `[[路径已隐藏]]` ... `[[路径已隐藏]]` 括号包裹：





【路径已隐藏】

崩溃=孤儿锁可检测

EVIDENCE

摘要器的 KV cache 杀手锏：摘要调用不是「新开一个对话让小模型总结」，而是重放会话自己的 system prompt + tools + 消息前缀，把压缩指令作为最后一条 user message 追加——这使得辅助调用成为最后一次路由请求的真前缀，provider 的前缀 KV cache 直接复用而不是失效。压缩指令本身要求输出固定的 8 段 Markdown 结构（Primary Request / Key Concepts / Files and Code / Errors and Fixes / Pending Jobs / Current Work / Next Step / Critical Context）。

[[路径已隐藏]] · COMPACTION_INSTRUCTION

EVIDENCE

四条硬不变式 (源码级):

1. **Shrink 不变式**：框架后的摘要 token 必须 < 被影蔽内容 token，否则抛错拒绝提交；
2. **边界配对**：`toolPairingBalancedBefore/After` 保证不拆开 assistant 的 tool-call 与 tool/result 配对；
3. **锁语义**：锁在 start 时同步写入，崩溃留下「孤儿锁」——可检测，而不是误报完成；
4. **影子定价**：`[[路径已隐藏]]` 与 `[[路径已隐藏]]` 记录被替换范围的确切 token 数，纯消费者可直接扣减，无需保留每节点价格。

[[路径已隐藏]] · region.ts · [[路径已隐藏]]

1.3 配套子系统

子系统	ctx key	职责	关键设计
Token Meter	ctx.tokenMeter	请求压力与逐节点定价	provider usage 锚点复用（同 envelope 才复用），否则保守启发式；O(surface) 快照
Tool-Result Pruner	ctx.toolResultPruner	超预算工具结果的无模型剪枝	head 4096 + 固定省略标记 + tail 1024, 阈值 8192 码点；二次扫描零改写（幂等）

子系统	ctx key	职责	关键设计
Spill	<code>ctx.spillStore</code>	超大工具输出落盘	内联结果替换为有界预览 + 检索定位符，原文持久化
Session Reference	<code>ctx.sessionReferenceResolver</code>	跨会话引用快照	有界 ($\leq 3 \times 64\text{KB}$)、只读投影，注入时机在 pre-step claim 或 inject
Context 扩展	—	time / tmux / agent-instructions	「不定义工具也进请求」的模型可见上下文，按需 opt-in

02 在 Agent Loop 中的作用

上下文管理不是 Loop 外挂的清理工，而是嵌在 Loop 骨架的两个钩子 + 一个命令入口上，决定「这一步发什么给模型」。

2.1 三个触发时机

● agent/pre-step · serial waterfall

每步请求推导之前运行。压力触发：tokenMeter 测量 $\geq$ 阈值 ($\text{thresholdRatio} \times$ 上下文窗口) $\rightarrow$ pruner 先行 $\rightarrow$ 仍超则选区摘要。返回 reject 可让 turn 闭合但不消耗 step——「明知要重试就不发请求」。

● agent/request-error · context-overflow

provider 确认上下文溢出后触发。绕过常规阈值强制压缩；`maxOverflowRetries` 封顶；只有 `replaceGeneration` 前进了才返回 retry——「没有真实进展就保留原始错误，不假装恢复」。

● /compact 命令 · runMaintenance

用户手动触发（低于阈值也行）。走 agent 空闲维护通道：与 turn 串行化，摘要期间的 inject 可以落在 marker 之间，只有选中 span 必须稳定。

KV-CACHE 视角 (dsh 文档显式建模)

append-only 事件保持前缀可复用；`replace` 从第一个被影蔽的消息起使缓存失效。这就是摘要器要「重放前缀」的原因——把压缩本身也设计成缓存友好的。上下文管理的每一步都同时回答两个问题：模型看到什么和缓存还剩多少。

失败也要可回放

手动压缩的六类失败 `busy / cancelled / changed / summary / commit / persistence` 各自有语义；失败尝试仍然写进日志——「宁可失败也不假装成功」是 dsh 的工程信条。

03 MemGPT / Letta · 虚拟上下文管理

arXiv:2310.08560 (UC Berkeley, 2023) → Letta (YC W24, ~22K★)。一切的起点：把 OS 内存分层搬进 LLM。

CORE THESIS

上下文窗口 = RAM，外部存储 = 磁盘，Agent 自己就是内存管理器——分页（page in/out）由模型自主调用工具完成，而不是框架替它决定。

MemGPT / Letta · 虚拟上下文管理 (OS 内存分层)

论点：把上下文窗口当 RAM、外部存储当磁盘，Agent 自己是内存管理器——分页由模型自主调用工具完成

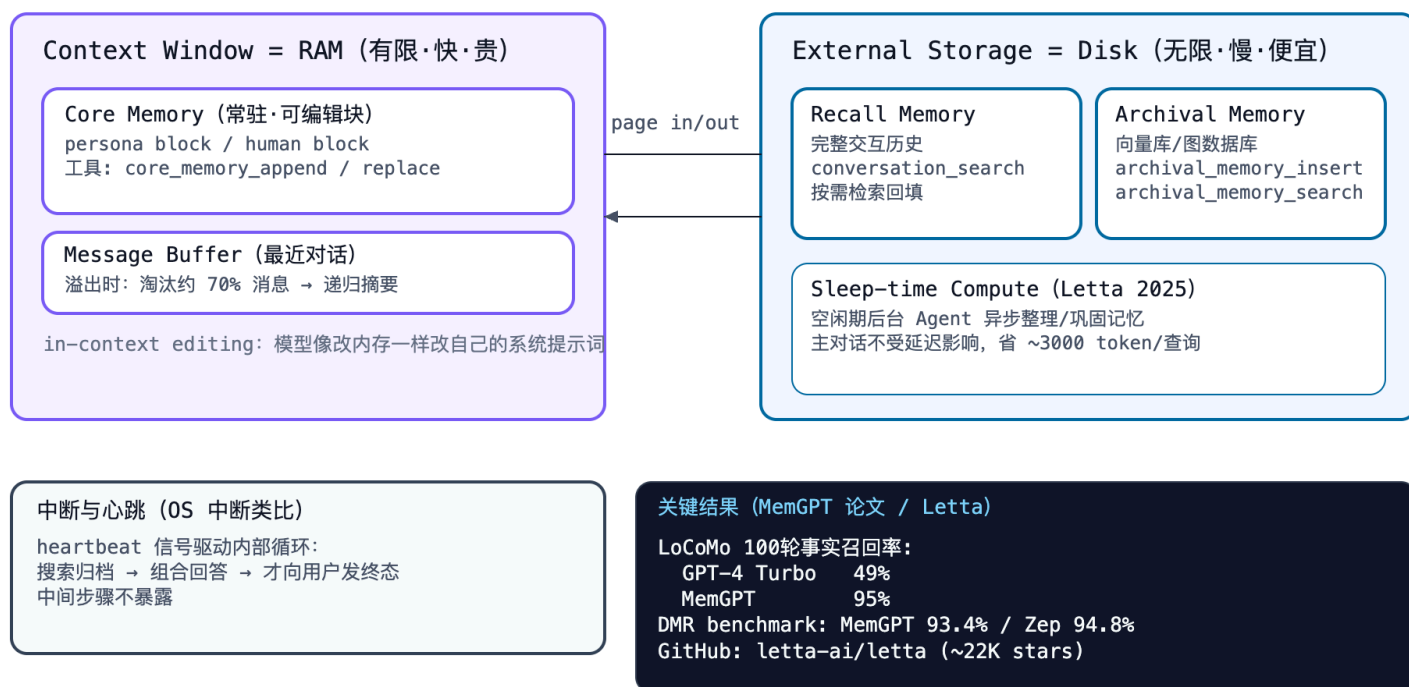


图 2 · MemGPT/Letta 三层记忆 + 分页工具 + sleep-time compute (Excalidraw)

源文件: context-diagrams/02-memgpt-letta.excalidraw

- **Core Memory:** 常驻上下文的可编辑块 (persona/human)，模型用 `core_memory_append/replace` 直接改自己的「系统提示词」——in-context editing
- **Recall Memory:** 完整交互历史 + `conversation_search` 按需检索回填
- **Archival Memory:** 向量库/图库冷存储，`archival_memory_insert/search` 分页换入换出
- **中断与心跳:** heartbeat 信号驱动内部循环 (搜档案→组合答案→才给用户终态)，中间步骤不暴露

- **Sleep-time Compute** (Letta 2025): 空闲期后台 Agent 异步巩固记忆, 主对话零延迟, 实测省 ~3000 token/查询



CRITIQUE

MemGPT 的代价: 每步都多出记忆工具的决策开销, 模型要「边干活边当内存管理器」, 弱模型管不好自己的内存; 分层抽象对短任务纯属过度设计。dsh 反其道行之——框架管内存 (策略在 pre-step), 模型只管干活; Letta 则把两者折中为「sleep-time 专职管理 Agent」。

04 Context-Folding · 可学习的上下文折叠

arXiv:2510.11967 (ByteDance Seed + CMU + Stanford, 2025)。上下文管理从「框架策略」升级为「模型的认知技能」，用 RL 端到端训练。

CORE THESIS

不是等上下文满了再被动摘要，而是 Agent 主动 `branch()` 出独立上下文的子轨迹、干完活 `return()` 把中间步骤全部折叠，只带回结论——分解与折叠是被 RL 训出来的认知技能。

Context-Folding · 把上下文管理变成可学习的认知技能

论点：不是等上下文满了再被动摘要，而是 Agent 主动 `branch` 出子轨迹、完成后 `fold` 掉中间步骤，只带回结论



推理时通过 KV-cache 回滚实现折叠 — 主轨迹前缀缓存不失效，几乎零成本

FoldGRPO (端到端 RL)

- 密集 token 级过程奖励：奖励有效分解 + 上下文管理
- 训练出的折叠行为：更难任务 → 更多分支 (自适应)
- 长度泛化：能处理比训练时更多分支的任务

结果 (Seed-OSS-36B)

BrowseComp-Plus: 62.0% (32K 预算)
SWE-Bench Verified: 58.0%
≈ 超越 327K ReAct 基线
显著优于 Summary Agent
上下文压缩率 > 90%
主轨迹长度 ~8K

图 3 · Context-Folding: branch/return 折叠机制 + FoldGRPO 过程奖励 (Excalidraw)

源文件：context-diagrams/03-context-folding.excalidraw

- 两个原语：`branch(description, prompt)` 创建子轨迹 (Execute 状态)，`return(message)` 折叠子轨迹只保留摘要 (回到 Plan 状态)
- 形式化：Agent 策略变为 $\pi(a | q, F(\tau < i))$ —— F 是上下文管理器，对历史做折叠变换

- **FoldGRPO**：密集 token 级过程奖励，奖励有效分解 + 上下文管理；推理时用 **KV-cache 回滚**实现折叠，主轨迹前缀缓存不失效
- **自适应行为**：更难的任务 → 更多分支与工具调用；能泛化到比训练时更长的任务

结果 (Seed-OSS-36B)

BrowseComp-Plus	62.0% (32K 预算)
SWE-Bench Verified	58.0%
上下文压缩率	>90%，主轨迹 ~8K

与 dsh 对照

dsh 的 compaction 是**框架策略**（阈值驱动、事后压缩、保留尾窗口）；Folding 是**模型技能**（预判性分支、结构化折叠、RL 优化）。两者可叠加：框架管底线，模型做优化。dsh 的 subagent/code mode 已有「子调用独立上下文」的雏形，差的是训练。

05 ACON · 失败驱动的压缩提示优化

Agent Context Optimization (arXiv 2025-10)。解决的问题是：压缩提示词 (prompt) 该保留什么，不该靠人拍脑袋。

CORE THESIS

用「全量上下文成功 vs 压缩上下文失败」的配对轨迹自动学出压缩指南：失败在哪，就往压缩提示里写「这类信息以后必须保留」。梯度无关，兼容闭源模型 API。

ACON · 失败驱动的上下文压缩提示优化

论点：压缩提示不该靠人拍脑袋—用「全量上下文成功 vs 压缩上下文失败」的配对轨迹自动学出该保留什么

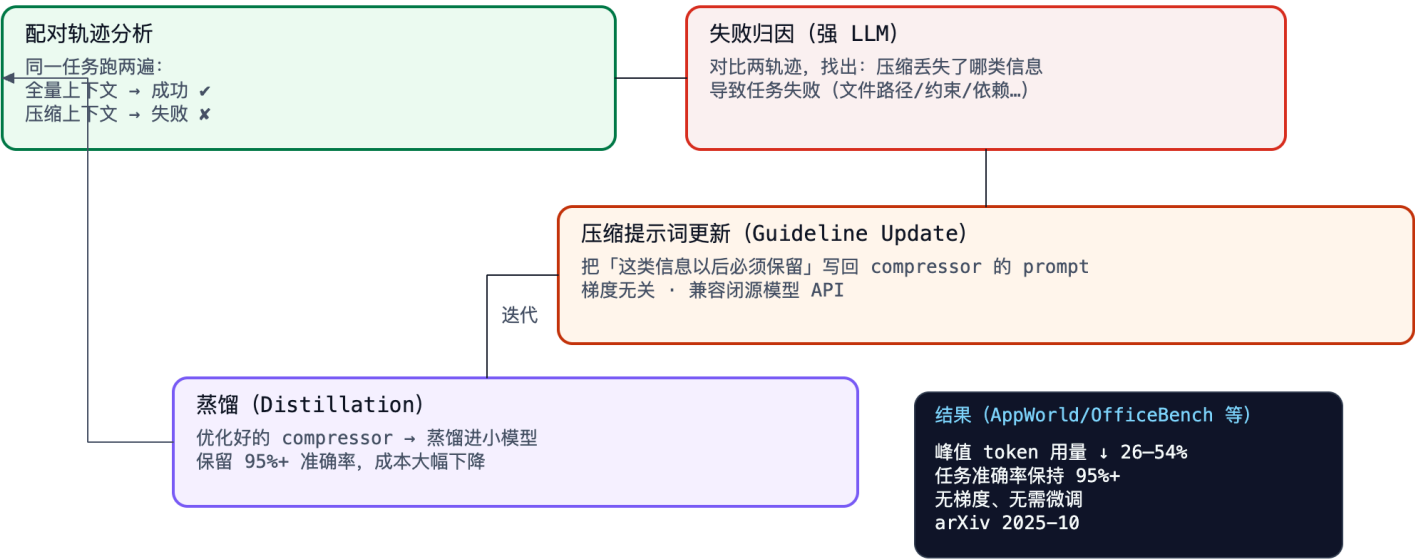


图 4 · ACON：配对轨迹分析 → 失败归因 → 压缩指南更新 → 蒸馏 (Excalidraw)

源文件：context-diagrams/04-acon.excalidraw



- AppWorld / OfficeBench / 多目标 QA 上：峰值 token ↓ 26-54%，准确率保持 95%+
- 对 dsh 的直接启示：dsh 的 `COMPACTION_INSTRUCTION` 是手写固定的 8 段结构——完全可以用 ACON 思路对自己的会话日志做离线优化，而 dsh 的日志恰好提供了配对轨迹所需的全部原料

06 综述与其余前沿速览

A Survey of Context Engineering for LLMs (arXiv:2507.13334, 1400+ 篇论文) 给出领域的统一分类法；其余论文各攻一个侧面。

Context Engineering 综述 · 1400+ 篇论文的统一分类法 (arXiv:2507.13334)

论点：上下文工程是超越提示工程的正式学科——系统性优化送入 LLM 的信息载荷

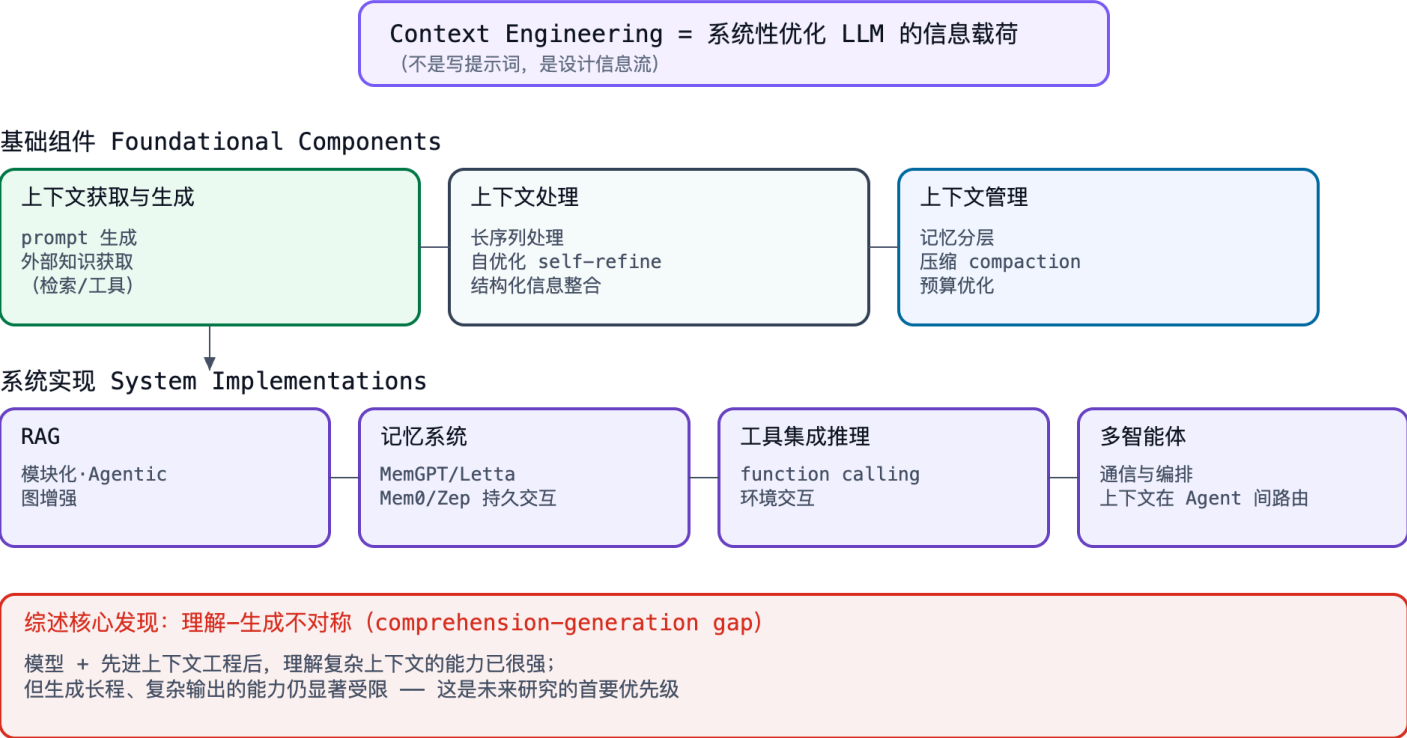


图 5 · 综述分类法：三大基础组件 × 四类系统实现 + 理解-生成不对称缺口 (Excalidraw)
源文件：context-diagrams/05-context-engineering-survey.excalidraw

6.1 其余重要论文一览

论文	核心主张	数据
The Complexity Trap arXiv:2508.21433	对 SWE Agent，简单遮蔽旧观察（observation masking）效果 ≈ 甚至优于昂贵的 LLM 摘要——价值在于去噪，不在精炼	OpenHands/Cursor 场景验证
ReSum arXiv:2509.13313	周期性「摘要-重置」让搜索 Agent 越过上下文墙（ReAct 会被墙挡住）	搜索类 benchmark +6.5%
Focus arXiv:2601.07190	仿黏菌探索策略：Agent 自主决定何时把学习固化进 Knowledge 块、何时剪掉原始轨迹（类 slime mold：探完死路就收缩，留化学标记）	SWE-bench Lite：token ↓ 22.7%，准确率持平
CAT: Context as a Tool arXiv:2512.32088	把上下文管理提升为 Agent 决策循环里的可调用工具（主动压缩代理权）	长程 SWE Agent
ComprExIT / E-mem / ContextWeaver / CMV 2026 系列	独立收敛到同一结论：LLM 摘要式压缩是「破坏性去语境化」——应改为 保结构 （依赖图/DAG/情节重建），需要时按需重导出	多个独立团队互证
Slipstream arXiv:2615.08570	压缩异步化 + 验证候选摘要保留了 Agent 的前向意图才提交	长程 Agent

FIELD CONSENSUS 2025-2026

摘要式压缩有天花板：丢失的不是 token，是结构。四个独立团队（ComprExIT、E-mem、ContextWeaver、CMV）在 2026 年收敛到「保结构 + 按需重建」；The Complexity Trap 证明一半的价值只是去噪。这意味着下一代的 compaction 不该是「更好的摘要提示」，而是「可重建的影蔽 + 可验证的保留」。

07 工业界与开源格局

Claude Code 的用户侧套路已经成熟成方法论；开源记忆层形成了清晰的路线分野。

Claude Code 上下文管理 · 工业界实战套路

论点：上下文是稀缺资源要主动经营—检查预算→定向压缩→彻底清空→永不丢失的外部锚点

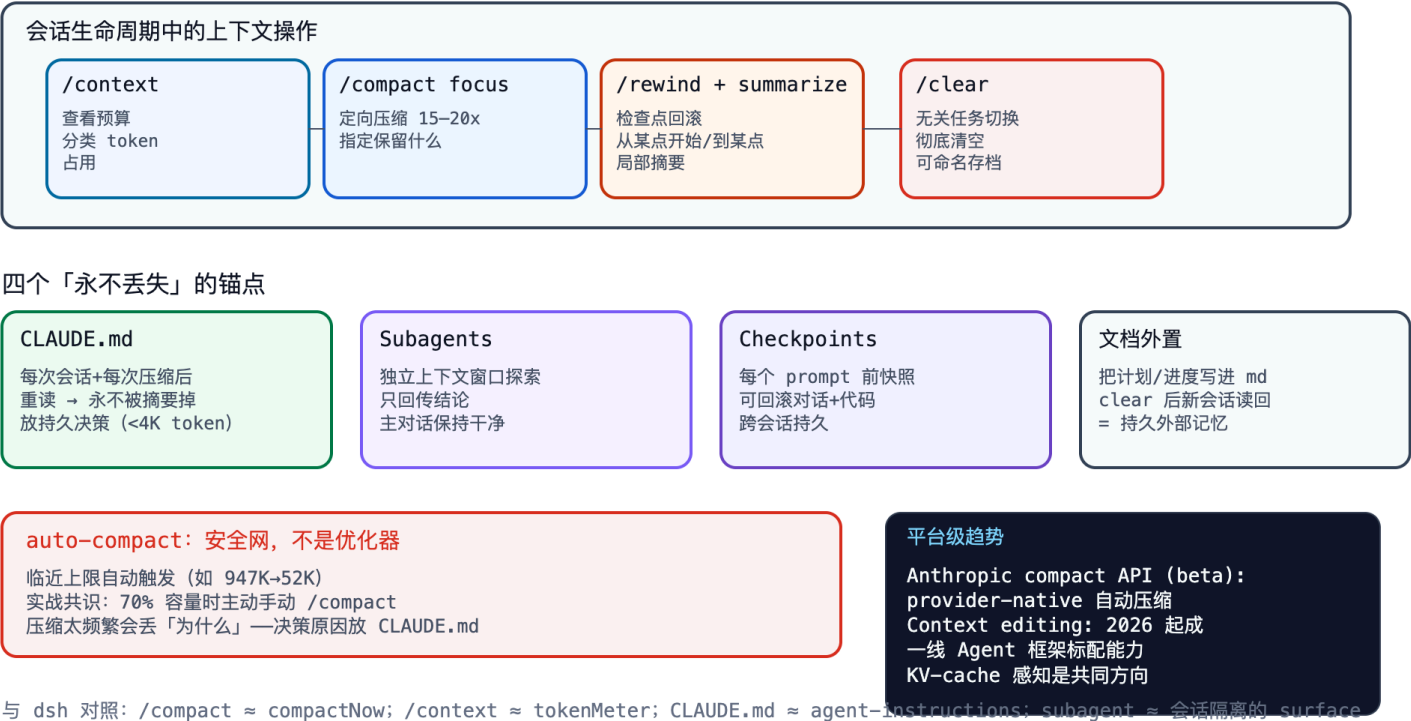


图 6 · Claude Code: 生命周期操作 + 四个「永不丢失」的锚点 (Excalidraw)

源文件: context-diagrams/06-claude-code-practice.excalidraw

7.1 Claude Code 实战套路（与 dsh 概念对照）

Claude Code	作用	dsh 对应物
<code>/context</code>	预算可视化：分类 token 占用	<code>ctx.tokenMeter</code> 测量
<code>/compact [focus]</code>	定向压缩 15—20×，可指定保留什么	<code>compactNow</code> + 手动命令
<code>/rewind + summarize</code>	检查点回滚 + 从某点局部摘要	Session fork / boundary
<code>/clear</code>	无关任务彻底清空	(dsh 哲学：换会话而非清空)
CLAUDE.md	每次压缩后重读 → 永不被摘要掉的锚点	agent-instructions (workspace 上下文)
Subagents	独立上下文探索，只回传结论	subagent + Code Mode 子调用
auto-compact	安全网：947K→52K 级自动压缩	pressure 触发的 compactIfNeeded

实战共识

70% 容量时**主动**手动压缩，别等 auto-compact 安全网（被动压缩质量更差）；压缩过频会丢「为什么」——决策原因放进 CLAUDE.md 这类永不被摘要的锚点。Anthropic 已把 compaction 做成平台 API (beta) ——provider-native 压缩正在成为标配。

7.2 开源记忆层路线图（2026.6 star 数据）

项目	Stars	路线	一句话
Mem0	~58K	抽取事实 + 多信号检索	向量+图+KV 混合；LLM 自编辑记忆 (ADD/UPDATE/MERGE/DELETE)
Zep / Graphiti	~27K	双时间知识图谱	每条事实带事件时间+摄入时间；时间推理强于 Mem0 约 15pp
Letta	~23K	OS 式分层运行时	记忆即架构；sleep-time compute；memory subagents
Cognee	~19K	ECL 管道知识图谱	add→cognify→memify→search；本地优先
MemOS	~9.5K	记忆操作系统	跨任务技能复用 + 记忆自进化，号称省 35% token

08 最佳实践总结

综合源码精读 + 9 篇论文 + 工业实践，今天就应该照做的十条。

- 01 日志是唯一事实，上下文只是投影。任何进模型请求的内容都能从日志重建（dsh 不变式）。messages 数组永远是推导结果，不是存储。
- 02 先去噪，再摘要。无模型的剪枝（head/tail 保段、观察遮蔽、spill 落盘）成本近零、无语义损失，先跑；LLM 摘要是最后手段（The Complexity Trap 证了一半价值只是去噪）。
- 03 压缩必须是事务。锁（start/end 括号）+ shrink 不变式（摘要必须严格更小）+ 边界配对（不拆 tool-call/result）+ 崩溃可检测（孤儿锁）。
- 04 KV cache 是第一公民。摘要调用重放会话前缀使缓存命中；replace 从第一个被影蔽消息起算失效范围。上下文决策永远同时考虑 token 和缓存。
- 05 测量用 provider usage 锚点。同 envelope 复用真实 usage，否则保守启发式——别相信纯字符数估算（dsh tokenMeter 的教训）。
- 06 结构化 checkpoint 优于自由摘要。固定 8 段结构（dsh）或四字段锚点（Factory：intent/changes/decisions/next）在多轮压缩后保持更高保真。
- 07 永不被摘要的锚点。CLAUDE.md / core memory block / agent-instructions——关键决策放锚点，摘要只处理过程性历史。
- 08 子上下文隔离。探索性、token 密集的操作（读大量文件/搜索）卸载到 subagent 或 branch，只回传结论——Claude Code 与 Context-Folding 在工业和学术两端同时验证。

09 70% 主动，别等安全网。被动 auto-compact 发生在最坏时机、保留质量最差；在阶段边界主动压缩并指定保留重点。

10 压缩质量可优化、可验证。ACON 用失败轨迹迭代压缩指南；Slipstream 验证摘要保留前向意图——把 compaction 当作可测试、可回归的组件，而不是玄学提示词。

09 未来发展方向

从 2025—2026 的论文收敛方向外推，五个正在发生的转变。

① 从框架策略 → 模型技能

Context-Folding / CAT / Focus 共同指向：上下文管理从 harness 的 if-else 变成模型可调用的工具与可 RL 训练的行为。未来的模型会「自带工作记忆管理能力」，框架退化为底线保障。

② 从摘要 → 保结构影蔽

四个 2026 团队独立收敛：摘要丢失的是结构。下一代 compaction = 依赖图/DAG 状态 + verbatim 保留 + 按需重导出 (re-derive on demand)，压缩变得可验证、可逆。

③ Provider-native 压缩

Anthropic compaction API 开了头。压缩终将下沉到推理层：KV-cache 感知的选择性驱逐、attention sink、上下文编辑成为模型服务的一等能力，框架只提供策略接口。

④ 预算约束的序贯决策

ContextBudget/BACM-RL 把上下文管理形式化为 token 预算下的序贯决策问题，用课程 RL 训练 curator。「每一步该花多少上下文预算」成为显式优化目标。

⑤ 理解-生成不对称的弥合

综述的核心发现：模型理解复杂上下文已强，生成长程复杂输出仍弱。上下文工程的下半场是「帮模型输出」——checkpoint 不仅是给下一轮读的，更是给模型自己续写用的脚手架。

⑥ 记忆的分层经济学

Mem0（抽取）/ Zep（时序图）/ Letta（运行时）/ MemOS（技能复用）各占一个生态位；sleep-time 异步巩固把记忆管理的延迟成本移出热路径。选型看问题形状，不看 star 数。

10 构建领先上下文管理系统的蓝图

如果今天从零构建一个领先的 Agent 上下文管理系统，把以上全部教训压成一张设计图。

DESIGN PRINCIPLE

五层设计：日志层（真相）→ 影蔽层（预算）→ 锚点层（永不丢失）→ 技能层（模型自主管理）→ 优化层（数据驱动的持续改进）。dsh 完成了前两层半，论文们指出了后两层。

- 01 第 1 层 · 事件溯源日志。**append-only、lossless、全量事件（含压缩事务本身）。模型可见内容 100% 可重建。影子定价协议让任何消费者能独立核算。这是 dsh 已经做对的部分，照抄。
- 02 第 2 层 · 三级减压管线。**①无模型：观察遮蔽 + head/tail 剪枝 + spill 落盘（先跑，零成本）；②结构化：依赖图影蔽，verbatim 保留 + 按需重导出（替代纯摘要）；③语义摘要：仅在 ①② 之后仍超预算时触发，输出固定结构 checkpoint，事务保护 + shrink 不变式。
- 03 第 3 层 · 锚点与预算。**常驻可编辑锚点块（persona/项目决策/用户偏好——CLAUDE.md 的泛化）；provider usage 锚点测量；KV-cache 失效范围作为每步决策的显式输入；70% 阈值主动触发 + 用户可解释的预算面板。
- 04 第 4 层 · 模型自主管理。**提供 branch()/return() 原语与 context 管理工具（CAT 思路），让强模型预判性地卸载子任务；sleep-time 后台 Agent 异步巩固记忆（Letta 思路）；策略兜底保住弱模型。可选用 FoldGRPO 式过程奖励微调。
- 05 第 5 层 · 数据驱动优化。**所有压缩决策都留下配对轨迹（全量 vs 压缩的结果对比）；离线跑 ACON 式失败归因迭代压缩指南；上线前用 Slipstream 式「前向意图保留」验证。compaction 成为一个有回归测试、有评测集的正常工程组件。
- 06 贯穿 · 缓存经济学。**每个 API 设计都回答三个问题：模型看到什么？缓存还剩多少？崩溃后能恢复吗？——前缀复用优先于重新构造，append 优先于 replace，replace 时从第一个变更点精确定价失

效。

MVP 路线建议

第一周：事件日志 + deriveMessages + tokenMeter（dsh 模式）。第二周：pruner + spill + 事务化 compaction。第三周：锚点块 + 前缀缓存友好的结构化摘要。之后：子上下文 branch 原语 → 配对轨迹采集 → 离线压缩指南优化。每一层都可独立回退，日志永远是底线。

11 开源上下文管理插件精读

把 00 章「上下文工程」里提到的开源项目集中精读：先看把上下文工程做成一套按需加载技能库的 Agent-Skills-for-Context-Engineering，再看 Mem0、Zep、Letta、Cognitive、A-MEM、LangMem 等记忆插件。每个项目的核心逻辑均来自源码，而非 README 宣传语。

11.1 Agent-Skills-for-Context-Engineering：把上下文工程做成一套按需加载的技能库

在逐一精读下面这些记忆插件之前，先看一个层级更高的开源项目——它回答的不是「记住什么」，而是「如何把一整套上下文工程的知识，作为技能按需装进 Agent，又不让上下文被撑满」。这是开发者 muratcankoylan 开源的 Agent-Skills-for-Context-Engineering（MIT，约 18K★，v2.5.0），把「上下文工程」这门手艺沉淀成 17 个可独立加载的 Agent Skill。它被北京大学《Meta Context Engineering via Agentic Skill Evolution》（2025）与 CMU、Yale 等多家合作的《Agent Harness Engineering: A Survey》（2026）两篇论文引用，被视为「静态技能架构」的代表作。

先解释术语：Agent Skill 指一个自包含的技能包，结构为一个目录 + 一个 SKILL.md 文件（开头用 YAML 声明 name / description，正文是 Markdown 指令）+ 可选的 references/（参考文档）与 scripts/（可执行脚本）。这正是 0.3 节「Skill 发现」模块最成熟的落地形态。

它的核心价值，是把上下文工程拆成一门有体系的学科。17 个技能分五类，每一类回答一个层次的问题：

类别	技能	回答的问题
基础	context-fundamentals · context-degradation · context-compression	上下文是什么、怎么退化、怎么压缩
架构	multi-agent-patterns · memory-systems · tool-design · filesystem-context · long-horizon-prompting · hosted-agents	系统结构怎么搭
运营	context-optimization · latent-briefing · evaluation · advanced-evaluation · harness-engineering · self-improvement-loops	上线后怎么优化、怎么评测、怎么自治
方法论	project-development	LLM 项目怎么从 0 到 1
认知	bdi-mental-states	怎么形式化建模 Agent 的心智状态

它管理上下文的方式，核心是 **渐进式披露 (progressive disclosure)**：每个 SKILL.md 开头只有两行 YAML (name + description, 约 50—100 token)，启动时系统提示词里只装这 17 个技能的名字 + 一句话描述——刚好够模型判断「何时该用哪个」。只有当任务真正触发某个技能时，模型才用文件读取工具把它的完整正文加载进上下文；再往深，才去读 references/ 或执行 scripts/。这种「先目录、后正文」的结构，把静态常驻成本从 $O(n)$ (全部技能全文) 降到 $O(1)$ (只有名字清单)。

具体怎么降开销，项目在 context-optimization 技能里给了四条有量化目标的路径 (读自源码)：

- **观察遮蔽 (observation masking)**：把冗长的工具输出换成紧凑引用 (形如 `[0bs:ref_id elided. Key: 摘要. 全文可检索]`)，目标 60—80% 减少，原文可随时取回；
- **压缩 (compaction)**：上下文利用率超过 70% 就触发，先压工具输出 (往往占 80%+ token)、再压旧轮次、再压检索文档，目标 50—70% token 减少、质量损失 <5%；
- **前缀缓存优化 (KV-cache optimization)**：稳定内容 (system prompt、工具定义) 放前缀、动态内容放末尾，命中率 70%+ 时成本降 50%+、延迟降 40%+；
- **上下文分区 (context partitioning)**：任务拆给子代理、各自隔离上下文。

还有两个更前沿的设计：**filesystem-context** 把文件系统当作上下文的溢出层——工具输出、计划、日志落盘，上下文里只留引用，把 $O(n)$ 静态成本转成按需读取；**latent-briefing** 在 worker 的 KV cache 里做表示层压缩，用「任务引导的注意力匹配」只保留当前子任务最相关的位置，无需把 orchestrator 的完整轨迹以文本重放。这一整套，本质上是把 0.4 的 KV cache 命中率、0.5 的 lost-in-middle 都变成了可执行的操作手册。

除了降开销，它还有几个值得注意的设计：

- **路由边界清晰**：每个技能都有「When to Activate」(何时激活) 与「Do not activate for」(何时不激活) 两段，明确写清与相邻技能的分工——17 个技能构成一张互相引用的「技能网络」，而非一份平铺的清单；
- **测量优先**：反复强调「measure before optimizing」(先测量、再优化、再验证效果)，每个技巧都给出可检验的量化阈值；
- **反直觉陷阱清单 (Gotchas)**：每个技能末尾列出一串陷阱，例如「系统提示词里放时间戳会摧毁缓存命中率」「前缀里一个空格变化就让整块 KV cache 失效」「上下文压力过大时做压缩反而会丢关键状态」；
- **出处追踪**：关键结论用 claim-xxx 标记挂到具体论文 (如 Liu et al. 2023 Lost in the Middle、RULER benchmark)，把「经验」与「有论文背书的结论」分开；
- **范式升级**：harness-engineering 把 Agent 本身当作一个需要「锁定指标、持久日志、新奇性检查、回滚规则、人类审批边界」的自治系统来设计，self-improvement-loops 进一步把 harness 本身当作优化目标——从「提示词工程」迈向「harness 工程」。

Agent-Skills-for-Context-Engineering: 17 个技能 + 渐进式披露

把上下文工程做成按需加载的技能库—先目录、后正文，静态常驻成本从 $O(n)$ 降到 $O(1)$

核心机制 · 渐进式披露 (Progressive Disclosure): 内容在被访问前不占任何 token



17 个技能 · 分五类，各自独立加载、互相引用



降开销的路径: 渐进式披露 + 观察遮蔽 (60–80%) + 压缩 (50–70%) + 前缀缓存 (命中率 70%+) + 上下文分区
上下文质量 > 数量: 只保留高信号 token, 其余按需取回—直接呼应 0.4 的 KV cache 命中率

图 11-1 · Agent-Skills-for-Context-Engineering: 17 个技能分五类 + 渐进式披露三级加载
源文件: context-diagrams/00-agent-skills-for-context-engineering.svg

它与接下来要精读的内存插件是互补关系：内存插件装「事实与状态」，这个项目装「能力与方法」——前者回答「记住什么」，后者回答「怎么把知识高效装进上下文」。这也是为什么把它放在插件之前：先建立「按需加载」的框架，再看各插件如何在这个框架里各自解决「记住什么」。

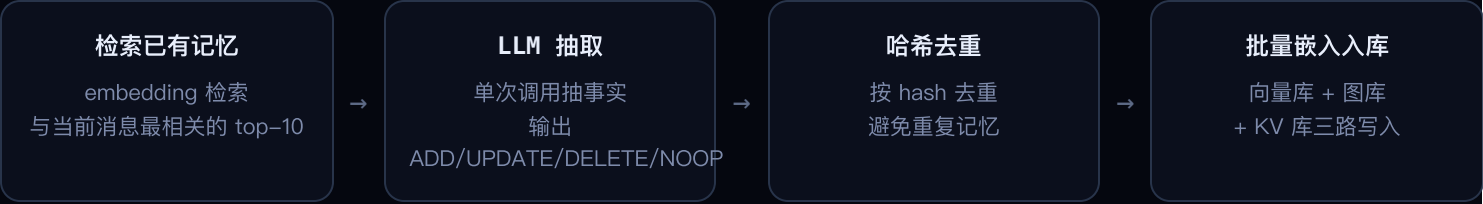
11.2 热门上下文管理插件：核心逻辑精读

下面按「问题形状」逐一精读当前最热门的开源上下文管理项目。它们的核心逻辑均来自源码，而非 README 宣传语。

项目	Stars	核心逻辑	针对的问题形状
Mem0	~58K	抽取式记忆 + 自适应更新 (ADD/UPDATE/DELETE/NOOP)	个性化：记住用户偏好、跨会话识别
Zep / Graphiti	~27K	时序知识图谱（事件时间 + 摄入时间）	时序推理：「谁在何时拥有什么」
Letta / MemGPT	~23K	OS 式分层记忆（Core/Recall/Archival）	长期运行、需要自我改进的 Agent
Cognee	~19K	ECL 知识图谱管道（Extract-Cognify-Load）	研究型：非结构化数据→图谱
Voyager	~14K	技能库（Skill Library）+ 自动课程	长程任务：技能累积复用
A-MEM	~1.1K	Zettelkasten 记忆演化（笔记/链接/演化）	长对话：记忆结构自发演化
LangMem	生态	LangGraph 原生记忆 SDK（热路径 + 后台）	已用 LangGraph 的团队，零基础设施

Mem0（~58K★，Apache 2.0） — 抽取式记忆 + 自适应更新

核心逻辑（读自 『路径已隐藏』，V3 分阶段批处理管道），`add()` 内部按五步走：



关键设计：记忆不是「原始对话」，而是「抽取后的事实」；增删改是**显式操作**——当用户更正偏好时是 UPDATE 而不是新增一条冲突的记忆。这里的 **embedding（嵌入）** 指把文本映射成高维向量，语义相近的文本向量距离近，是向量检索的基础。

局限同样清晰：抽取式管线容易丢细节，记忆停留在「记住」层面；图能力（实体关系、多跳查询）在免费层不可用；时序推理弱（LongMemEval 仅 49%，对比 Zep 的 63.8%）。

Zep / Graphiti（~27K★） — 时序知识图谱

核心逻辑：把记忆建模为**时序知识图谱**，每条事实携带两条时间轴——**事件时间**（事实在现实中何时为真）与**摄入时间**（系统何时记录）。关键能力是回答「谁在何时拥有什么」这类时序问题（例如「Alice 在 2 月前是预算负责人，之后是 Bob」），这是平面向量检索做不到的。**知识图谱（knowledge graph）** 指用「实体—关系—实体」三元组组织的结构化知识，支持图遍历查询。代价是需要额外的图数据库基础设施。

Letta / MemGPT (~23K★) — OS 式分层记忆

核心逻辑：把操作系统的内存分层搬进 LLM，分三层——**Core Memory**（常驻上下文的可编辑块，模型可自己改写）、**Recall Memory**（完整交互历史，按需检索回填）、**Archival Memory**（向量库冷存储，分页换入换出）。最鲜明的设计是：**Agent 自己就是内存管理器**——分页（page in/out）由模型自主调用工具完成，而非框架替它决定。这一章的完整精读见第 03 章。

Cognee (~19K★) — ECL 知识图谱管道

核心逻辑：**ECL 管道（Extract-Cognify-Load）**——把原始文档与对话经过「认知化（cognify）」转化为带类型的知识图谱，本地优先。它模糊了 RAG 与 Agent 记忆的边界，适合把非结构化数据沉淀为可查询的结构化知识。

A-MEM (NeurIPS 2025) — Zettelkasten 记忆演化

核心逻辑（读自 [\[路径已隐藏\]](#)）是三阶段：**笔记构建**（LLM 生成关键词/标签/上下文描述）、**链接生成**（embedding 检索近邻 + LLM 判断是否建立语义链接，形成「主题盒」）、**记忆演化**（新记忆触发对近邻旧记忆的 tags/context 更新）。关键差异：记忆从「只读」变为「可演化」——检索时不仅返回最像的一条，还带回它的链接邻居。**Zettelkasten（卡片盒笔记法）**是社会学家卢曼提出的笔记方法：每条笔记原子化、带编号、可互相链接，形成知识网络。A-MEM 的完整架构见图 0-4。

LangMem — LangGraph 原生记忆 SDK

核心逻辑：提供 `create_manage_memory_tool` / `create_search_memory_tool`，支持**热路径**（Agent 主动管理）与**后台**（自动抽取）两种模式，还能把用户反馈整合进系统提示词——这是**程序性记忆（procedural memory）**：记住「怎么做」（行为规则、方法），区别于记住「是什么」的语义记忆（semantic memory）与记住「经历过什么」的情节记忆（episodic memory）。